

PocketSmart AI

Your Smart Budget and Recommendation Assistant

Complete Project Documentation

Architecture, Working, Functions, Setup and Deployment Guide

Technologies: Python, FastAPI, Jinja2, SQLite, Google Gemini API

Table of Contents

1. Introduction
2. Problem Statement and Objectives
3. Key Features
4. Technology Stack
5. System Architecture and Workflow
6. Project Structure
7. Module and Function Explanation
8. Database Design
9. How a Recommendation Is Generated (Step by Step)
10. Security Measures
11. Setup and Run Guide (VS Code, Windows)
12. Testing
13. Troubleshooting
14. Uploading to GitHub
15. Limitations and Future Scope
16. Conclusion
17. Appendix: Source Code

1. Introduction

PocketSmart AI is a web application that gives smart, budget-based purchase recommendations. The user enters a total budget and a few details, and the application asks Google's Gemini AI model to suggest items that fit inside that budget. Every suggested item comes with its category, estimated price, a short reason, and a direct search link to a shopping or service platform such as Amazon, Flipkart, IKEA, Swiggy, Zomato or OYO.

The application covers three everyday planning areas: Home Interior, Party Planning and Jewelry. Users create an account, log in, use a planner, and can see all of their earlier recommendations in a history page.

2. Problem Statement and Objectives

People who plan a home makeover, a party or a jewelry purchase usually have a fixed budget but do not know how to divide it across items, and they spend a long time comparing options on many websites.

Objectives

- Give recommendations that stay within the budget entered by the user.
- Cover three areas: Home Interior, Party and Jewelry.
- Provide a ready link to the right platform for each item.
- Support an outfit photo in the Jewelry planner so suggestions match the colours.
- Store each user's history safely behind a login.
- Keep working (with default suggestions) even if the AI service is unavailable.

3. Key Features

Feature	Description
User accounts	Register and login with username and password. Passwords are stored as salted hashes.
Dashboard	Welcome page with the three planners and the five most recent recommendations.
Home Interior Planner	Inputs: budget, rooms, items with quantities, style. Output: furniture, lights, fans and decor within budget.
Party Planner	Inputs: budget, guest count, event type, venue. Output: split across catering, decoration and entertainment or venue.
Jewelry Planner	Inputs: budget, occasion, style, optional outfit image. Output: jewelry matching occasion and outfit colours.
Platform links	Each item gets a search link on the suggested platform.
History	All earlier requests and their results are saved per user.
Fallback mode	If Gemini fails, three default suggestions based on percentages of the budget are shown.

4. Technology Stack

Technology	Purpose
Python 3	Main programming language.
FastAPI	Backend web framework that defines the routes (URLs) and handles requests.
Uvicorn	Server that runs the FastAPI application on port 8000.
Jinja2	Template engine that fills HTML pages with data from Python.
HTML and CSS	Structure and design of the web pages.
SQLite (sqlite3)	Small file-based database that stores users and history.
Google Gemini API (google-genai)	AI model (default gemini-2.5-flash) that generates the recommendations.
python-dotenv	Reads secret values (API key) from the .env file.
python-multipart	Allows FastAPI to read form data and uploaded images.
itsdangerous	Signs the session cookie used to keep a user logged in.
hashlib, secrets	Standard Python modules used for password hashing and random salt.
VS Code and GitHub	Development environment and code hosting.

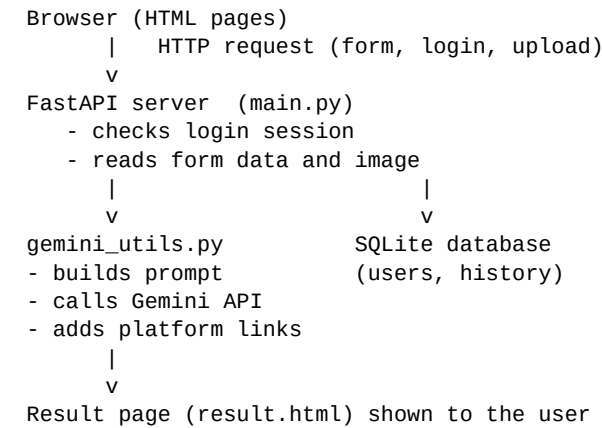
requirements.txt

This file lists the packages that must be installed. The command `pip install -r requirements.txt` installs all of them at once.

```
fastapi
uvicorn
jinja2
python-multipart
itsdangerous
python-dotenv
google-genai
```

5. System Architecture and Workflow

The application follows a simple three-layer design: the browser (frontend), the FastAPI server (backend logic), and external or stored data (SQLite database and Gemini API).



User journey

- Open the home page and press Get Started.
- Register an account, then log in. The username is stored in a signed session.
- The Dashboard opens. Choose Home, Party or Jewelry planner.
- Fill the form and submit. The server sends the details to Gemini.
- Gemini returns a JSON list of items. The server adds links and saves everything in history.
- The result page shows cards with item name, category, platform, price, reason and a link.
- The History page lists all past requests.

6. Project Structure

PocketSmartAI/	
main.py	Backend: routes, login, database, history
gemini_utils.py	AI logic: prompt, Gemini call, links, fallback
requirements.txt	List of packages to install
.env	Secret keys (never uploaded to GitHub)
pocketsmart.db	SQLite database (created automatically)
templates/	
base.html	Common layout, menu and styling
index.html	Home page with banner and planner cards
auth.html	Login and Register page (one template, two modes)
dashboard.html	Welcome, planners and recent recommendations
planner.html	Generic budget form used by all three planners
result.html	Recommendation cards
history.html	Full history of the logged-in user

The .env file

Variable	Meaning
GEMINI_API_KEY	Your key from aistudio.google.com/apikey. Without it the app shows fallback suggestions.
GEMINI_MODEL	Name of the Gemini model. Default is gemini-2.5-flash. Change it if the model is renamed.
SECRET_KEY	Any long random text. Used to sign the login session cookie.

7. Module and Function Explanation

7.1 gemini_utils.py (AI module)

Name	What it does
load_dotenv()	Loads values from the .env file into the environment.
MODEL, client	Reads the model name and creates the Gemini client with the API key.
LINKS	Dictionary that maps a platform name (amazon, flipkart, ikea, swiggy, zomato, oyo) to its search URL.
PLATFORMS	For each planner, the platforms the AI is allowed to suggest.
GOALS	For each planner, a one-line instruction describing what to recommend.

Name	What it does
fallback(kind, data)	Returns three default items worth 40%, 30% and 20% of the budget. Used when the AI call fails.
recommend(kind, data, image)	Main function. Builds the prompt, calls Gemini, parses the JSON, adds links, and returns the list of items.

How recommend() works, line by line in words

- Builds a prompt containing the planner goal, the user's inputs as JSON, the rule that the total price must not exceed the budget, the allowed platforms, and the exact JSON format required.
- If an outfit image was uploaded, it is attached to the request as image bytes with its MIME type.
- Calls client.models.generate_content with response_mime_type set to application/json so the reply is JSON.
- Removes any accidental code fences and reads the list under the key items.
- If anything goes wrong (no key, quota, network, bad JSON), the error is printed in the terminal and fallback() is used, so the app never crashes.
- For every item, takes the first platform word, finds its URL in LINKS, and adds the item name as a search query. If the platform is unknown, the link is #.

7.2 main.py (backend module)

Name	What it does
app, middlewares	Creates the FastAPI app. SessionMiddleware keeps users logged in. CORS middleware allows cross-origin requests.
lifespan / startup()	Runs once when the server starts and creates the users and history tables if they do not exist.
PLANNERS	Dictionary that defines each planner title and its form fields (name, label, input type). The same template draws every form from this data.
db()	Opens a SQLite connection that returns rows addressable by column name.
hash_pw(pw, salt)	Creates a random salt and hashes the password with PBKDF2-HMAC-SHA256 (100,000 rounds). Stores salt and hash together.
check_pw(pw, stored)	Re-hashes the typed password with the stored salt and compares it with the stored value.
me(request)	Returns the logged-in username from the session, or None.
render(request, name, **ctx)	Shortcut that renders a template and always passes the current user to it.

Routes (URLs)

URL	Method	Purpose
/	GET	Home page with banner and planner cards.
/register	GET, POST	Show the form; create a new user. Shows an error if the username exists.
/login	GET, POST	Show the form; verify the password and store the username in the session.

URL	Method	Purpose
/logout	GET	Clear the session and go to login.
/dashboard	GET	Welcome page with planners and the latest five recommendations.
/planner/{kind}	GET	Show the form for home, party or jewelry. Redirects to login if not logged in.
/generate-home, /generate-party, /generate-jewelry	POST	Receive the form, call recommend(), save to history, show the result page.
/history	GET	List every saved request of the current user, newest first.
/session-info	GET	Returns JSON showing whether a user is logged in.

make_route(kind)

Instead of writing three almost identical functions, `make_route` creates one handler for each planner. The handler checks the login, reads all form fields into a dictionary, reads the optional image file (bytes and content type), calls `recommend()`, inserts the request and result into the history table, calculates the total price, and renders `result.html`. A loop at the bottom registers the three POST routes.

7.3 Templates

Template	Role
base.html	Defines the page skeleton, CSS styles and the top menu. The menu changes depending on whether the user is logged in. All other pages extend it.
index.html	Hero banner, three planner cards and testimonials.
auth.html	One template used for both Login and Register. The variable <code>mode</code> changes the heading and button, and <code>error</code> shows a message.
dashboard.html	Greets the user, shows planner cards and recent recommendations.
planner.html	Loops over the field list of the chosen planner and draws each input. File fields get an image-only picker. The form uses multipart encoding so images can be uploaded.
result.html	Shows the budget, the estimated total and one card per item with a link button.
history.html	Lists every past request with its items, prices and platforms.

8. Database Design

SQLite stores data in a single file named `pocketsmart.db`, created automatically on first run.

Table: users

Column	Type	Description
id	INTEGER, primary key	Unique user number.
username	TEXT, unique	Login name. Duplicates are rejected.
pw	TEXT	Salt and PBKDF2 hash joined by a dollar sign. The real password is never stored.

Table: history

Column	Type	Description
id	INTEGER, primary key	Unique record number.
username	TEXT	Owner of the record.
kind	TEXT	home, party or jewelry.
inputs	TEXT (JSON)	The form values the user entered.
results	TEXT (JSON)	The list of recommended items.
created	TEXT	Timestamp added automatically.

9. How a Recommendation Is Generated (Step by Step)

- Step 1.** The user opens /planner/home and fills budget, rooms, items and style.
- Step 2.** The browser sends a POST request to /generate-home as a multipart form.
- Step 3.** The handler confirms the user is logged in; otherwise it redirects to /login.
- Step 4.** Form values are collected in a dictionary. If a file is attached, its bytes are read.
- Step 5.** recommend('home', data, image) builds the prompt and calls the Gemini API.
- Step 6.** Gemini replies with JSON: a list of items with name, category, platform, price and reason.
- Step 7.** The code parses the JSON and adds a search link to each item. On any error it uses fallback().
- Step 8.** The request and result are saved in the history table.
- Step 9.** The total price is added up and result.html shows the cards, the budget and the estimated total.
- Step 10.** The user can open History at any time to see the saved result.

Example of the JSON that Gemini returns

```
{
  "items": [
    {
      "name": "LED ceiling light",
      "category": "Lighting",
      "platform": "Amazon",
      "price": 2500,
      "reason": "Bright and energy efficient for a living room."
    }
  ]
}
```

10. Security Measures

- Passwords are never stored in plain text. PBKDF2-HMAC-SHA256 with a random salt and 100,000 iterations is used.
- The login session cookie is signed with SECRET_KEY, so it cannot be changed by the user.
- Planner pages and history require login; otherwise the user is redirected to the login page.
- History queries use the logged-in username, so a user sees only their own data.
- SQL statements use placeholders (?), which protects against SQL injection.
- The API key is kept in .env, and .env is excluded from GitHub by .gitignore.
- Jinja2 escapes values in templates by default, which protects against script injection.

11. Setup and Run Guide (VS Code, Windows)

A. Install the software (one time)

- Python 3.10 or newer from python.org. Tick Add python.exe to PATH during installation.
- Visual Studio Code from code.visualstudio.com.
- Git from git-scm.com (needed only for uploading with commands).

B. Create the project

- Create a folder named PocketSmartAI on the Desktop and open it in VS Code with File, Open Folder. Click Trust.
- Create a folder named templates inside it.
- Create the files exactly as shown in the Project Structure section and paste the code from the Appendix.
- Press Ctrl+S in every file to save.

C. Get the Gemini API key

- Open aistudio.google.com/apikey and sign in with a Google account.
- Click Create API key and copy it.
- In the .env file, write GEMINI_API_KEY= followed by the key, with no spaces or quotes.

```
GEMINI_API_KEY=your_key_here
GEMINI_MODEL=gemini-2.5-flash
SECRET_KEY=any-long-random-text-123456
```

D. Install packages and run

Open the VS Code terminal (Terminal, New Terminal) and run these commands one by one:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python main.py
```

If PowerShell blocks the activate command, run **Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass** and activate again. When the terminal shows *Uvicorn running on http://127.0.0.1:8000*, open that address in a browser. To stop the server press Ctrl+C.

12. Testing

No.	Test	Expected result
1	Open http://127.0.0.1:8000	Home page with banner and three planner cards.
2	Register a new username and password	Redirect to the login page.
3	Register the same username again	Error message: Username already taken.
4	Login with a wrong password	Error message: Invalid username or password.
5	Login with correct details	Dashboard opens with Welcome message.
6	Open /planner/home without login	Redirect to the login page.
7	Home planner with budget 50000	Result cards, with total not above the budget.

No.	Test	Expected result
8	Party planner with guests and event type	Items split across food, decoration and venue.
9	Jewelry planner with an outfit image	Jewelry suggestions matching the outfit.
10	Open History	All earlier requests are listed.
11	Remove the API key and generate	Default suggestion cards appear; the app does not crash.
12	Click Logout	Session is cleared and the Login link returns.

13. Troubleshooting

Problem	Reason and fix
python is not recognized / Python was not found	Python is not installed or not on PATH. Install from python.org and tick Add to PATH. Restart VS Code.
git is not recognized	Git is not installed. Install from git-scm.com and restart VS Code.
ModuleNotFoundError	The virtual environment is not active or packages are not installed. Activate venv and run pip install -r requirements.txt.
Cannot run activate script	Run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then activate again.
Default suggestion cards appear	API key missing or wrong, .env not saved, no internet, or quota reached. Read the line starting with Gemini error in the terminal.
Model not found error	Change GEMINI_MODEL in .env to a currently available model name.
Quota or 429 error	Free usage limit reached. Wait and try again.
Port already in use	Close the old server with Ctrl+C or change 8000 to 8001 in main.py.
File name not valid in VS Code	Type the file name by hand; do not paste it. Remove spaces.

14. Uploading to GitHub

Method 1: Website upload (no Git needed)

- Sign in at github.com, click +, then New repository. Enter a name such as PocketSmart-AI, choose Public, and create it.
- On the Code tab click Add file, then Upload files.
- Drag main.py, gemini_utils.py, requirements.txt and the templates folder from File Explorer into the page.
- Click Commit changes.
- Do not upload .env, venv, __pycache__ or pocketsmart.db. The .env file contains your secret key.

Method 2: From VS Code with Git

- Install Git and restart VS Code. Sign in through the Accounts icon, Sign in with GitHub.
- Create a .gitignore file containing the four lines shown below.

- Create an empty repository on GitHub and copy its address.

```
.env
venv/
__pycache__/
*.db

git config --global user.name "Your Name"
git config --global user.email "your-github-email@example.com"
git init
git add .
git commit -m "PocketSmart AI"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/PocketSmart-AI.git
git push -u origin main
```

Refresh the repository page to see the files. If an API key is ever uploaded by mistake, delete that key at aistudio.google.com/apikey and create a new one.

15. Limitations and Future Scope

Current limitations

- Prices are estimates produced by the AI, not live prices from the shopping sites.
- Links are search links, not direct product pages, because live scraping is not used.
- Quality of results depends on the Gemini model and the free quota.
- History cannot yet be deleted or exported.

Future scope

- Live price lookup through official shopping or affiliate APIs.
- Budget charts, saved favourites, and export of results to PDF.
- Password reset, email verification and JWT token login for a mobile app.
- Deployment to a cloud host so anyone can use the app online.
- More planners such as travel, wedding and electronics.

16. Conclusion

PocketSmart AI shows how a generative AI model can be combined with a lightweight Python web framework to solve a practical problem. The project covers user authentication, a database, image upload, an AI integration with a safe fallback, and a clean multi-page interface. It can be run locally in VS Code, stored on GitHub, and extended with further planners and live data in the future.

17. Appendix: Source Code

The two Python files are listed below. The HTML templates are provided in the project folder. Long lines wrap with a continuation marker; when typing the code, join wrapped lines into one line.

gemini_utils.py

```
import os, json, re
from urllib.parse import quote_plus
from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()
MODEL = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

# Real search links (no fake scraping) for each platform
LINKS = {
    "amazon": "https://www.amazon.in/s?k=", "flipkart": "https://www.flipkart.com/search?q=",
    "ikea": "https://www.ikea.com/in/en/search/?q=", "swiggy": "https://www.swiggy.com/search?query=",
    "zomato": "https://www.zomato.com/search?q=", "oyo": "https://www.oyorooms.com/search/?location=",
}
PLATFORMS = {
    "home": "Amazon, IKEA, Flipkart",
    "party": "Swiggy, Zomato, OYO, Amazon (for decor)",
    "jewelry": "Amazon, Flipkart",
}
GOALS = {
    "home": "Recommend furniture/lights/fans/decor for each room and quantity requested.",
    "party": "Split the budget across catering, decoration and entertainment/venue for the event type and guest count.",
    "jewelry": "Recommend jewelry matching the occasion and style. If an outfit image is given, match its colours.",
}

def fallback(kind, data):
    b = float(data.get("budget", 0) or 0)
    return [{"name": f"Budget {kind} pick {i+1}", "category": c, "platform": "Amazon",
            "price": round(b * p), "reason": "Default suggestion (AI unavailable)."}
            for i, (c, p) in enumerate([("Essential", .4), ("Comfort", .3), ("Extra", .2)])]

def recommend(kind, data, image=None):
    prompt = (f"You are a budget assistant for Indian shoppers. {GOALS[kind]}\n"
             f"User inputs: {json.dumps(data)}\nAll prices in INR. Total of all item prices MUST NOT exceed the budget.\n"
             f"Use only these platforms: {PLATFORMS[kind]}\n"
             'Return ONLY JSON: {"items":[{"name":"","category":"","platform":"","price":0,"reason":""}]}')
    contents = [prompt]
    if image:
        contents.append(types.Part.from_bytes(data=image[0], mime_type=image[1]))
    try:
        r = client.models.generate_content(model=MODEL, contents=contents,
            config=types.GenerateContentConfig(response_mime_type="application/json"))
        items = json.loads(re.sub(r"```json|```", "", r.text).strip())["items"]
        assert items
    except Exception as e:
        print("Gemini error:", e)
        items = fallback(kind, data)
    for i in items:
        p = str(i.get("platform", "")).split(",")[0].strip().split(" ")[0].lower()
        i["link"] = LINKS[p] + quote_plus(str(i.get("name", ""))) if p in LINKS else "#"
    return items
```

main.py

```
import os, json, sqlite3, hashlib, secrets
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request, Form
from fastapi.responses import RedirectResponse
from fastapi.templating import Jinja2Templates
from fastapi.middleware.cors import CORSMiddleware
from starlette.middleware.sessions import SessionMiddleware
from dotenv import load_dotenv
from gemini_utils import recommend

load_dotenv()
@asynccontextmanager
async def lifespan(app):
```

```

        startup()
        yield

app = FastAPI(title="PocketSmart AI", lifespan=lifespan)
app.add_middleware(SessionMiddleware, secret_key=os.getenv("SECRET_KEY", "dev-secret"))
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
templates = Jinja2Templates(directory="templates")
DB = "pocketsmart.db"

# Form fields per planner: (name, label, input type)
PLANNERS = {
    "home": ("Home Interior Planner", [("budget", "Total budget (Rs.)", "number"), ("rooms", "Rooms (e.g. Living Room,
        + Kitchen)", "text"),
        ("quantities", "Items & quantities (e.g. 4 lights, 2 fans, 1 dining table)", "text"), ("style", "Style (m
        + odern, minimal...)", "text")])),
    "party": ("Party Planner", [("budget", "Total budget (Rs.)", "number"), ("guests", "Guest count", "number"),
        ("event_type", "Event type (birthday, wedding...)", "text"), ("venue", "Venue details", "text")])),
    "jewelry": ("Jewelry Planner", [("budget", "Total budget (Rs.)", "number"), ("occasion", "Occasion", "text"),
        ("style", "Style preference", "text"), ("image", "Outfit image (optional)", "file")])),
}

def db():
    c = sqlite3.connect(DB); c.row_factory = sqlite3.Row; return c

def startup():
    with db() as c:
        c.execute("CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY, username TEXT UNIQUE, pw TEXT)")
        c.execute("CREATE TABLE IF NOT EXISTS history(id INTEGER PRIMARY KEY, username TEXT, kind TEXT, inputs TEXT, r
            + esults TEXT, created TEXT DEFAULT CURRENT_TIMESTAMP)")

def hash_pw(pw, salt=None):
    salt = salt or secrets.token_hex(8)
    return salt + "$" + hashlib.pbkdf2_hmac("sha256", pw.encode(), salt.encode(), 100000).hex()

def check_pw(pw, stored):
    return hash_pw(pw, stored.split("$")[0]) == stored

def me(request): return request.session.get("user")

def render(request, name, **ctx):
    return templates.TemplateResponse(request, name, {"user": me(request), **ctx})

@app.get("/")
def home(request: Request): return render(request, "index.html", planners=PLANNERS)

@app.get("/register")
def register_page(request: Request): return render(request, "auth.html", mode="register", error=None)

@app.post("/register")
def register(request: Request, username: str = Form(...), password: str = Form(...)):
    try:
        with db() as c: c.execute("INSERT INTO users(username,pw) VALUES(?,?)", (username, hash_pw(password)))
    except sqlite3.IntegrityError:
        return render(request, "auth.html", mode="register", error="Username already taken")
    return RedirectResponse("/login", 303)

@app.get("/login")
def login_page(request: Request): return render(request, "auth.html", mode="login", error=None)

@app.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    row = db().execute("SELECT pw FROM users WHERE username=?", (username,)).fetchone()
    if not row or not check_pw(password, row["pw"]):
        return render(request, "auth.html", mode="login", error="Invalid username or password")
    request.session["user"] = username
    return RedirectResponse("/dashboard", 303)

@app.get("/logout")
def logout(request: Request):
    request.session.clear(); return RedirectResponse("/login", 303)

@app.get("/session-info")
def session_info(request: Request): return {"user": me(request), "logged_in": bool(me(request))}

@app.get("/planner/{kind}")
def planner(request: Request, kind: str):
    if not me(request): return RedirectResponse("/login", 303)
    title, fields = PLANNERS[kind]
    return render(request, "planner.html", kind=kind, title=title, fields=fields)

def make_route(kind):
    async def handler(request: Request):
        if not me(request): return RedirectResponse("/login", 303)

```

```

        form = await request.form()
        data = {k: v for k, v in form.items() if isinstance(v, str)}
        img = form.get("image"); image = None
        if img is not None and getattr(img, "filename", ""):
            image = (await img.read(), img.content_type or "image/jpeg")
        items = recommend(kind, data, image)
        with db() as c:
            c.execute("INSERT INTO history(username,kind,inputs,results) VALUES(?,?,?,?)", (me(request), kind, json.dumps(data), json.dumps(items)))
        total = sum(float(i.get("price", 0) or 0) for i in items)
        return render(request, "result.html", title=PLANNERS[kind][0], items=items, total=total, budget=data.get("budget"))
    return handler

for k in PLANNERS:
    app.add_api_route(f"/generate-{k}", make_route(k), methods=["POST"])

@app.get("/dashboard")
def dashboard(request: Request):
    if not me(request): return RedirectResponse("/login", 303)
    rows = db().execute("SELECT kind,created,inputs FROM history WHERE username=? ORDER BY id DESC LIMIT 5", (me(request),)).fetchall()
    recent = [{"kind": r["kind"], "created": r["created"], "budget": json.loads(r["inputs"]).get("budget")} for r in rows]
    return render(request, "dashboard.html", planners=PLANNERS, recent=recent)

@app.get("/history")
def history(request: Request):
    if not me(request): return RedirectResponse("/login", 303)
    rows = db().execute("SELECT * FROM history WHERE username=? ORDER BY id DESC", (me(request),)).fetchall()
    data = [{"kind": r["kind"], "created": r["created"], "inputs": json.loads(r["inputs"]), "items": json.loads(r["results"])} for r in rows]
    return render(request, "history.html", rows=data)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="127.0.0.1", port=8000, reload=True)

```